The CPU

A very tall pile of simple

Ekansh Goenka

bytecolony.computer

print at 100% scale (“Actual Size”) — cut along the black line — true book size 6 × 9 in

Contents

Part 1: Foundations **1**
1.1. How to Read This 3
1.2. Circuits & Switches 3
1.3. Relays & Logic Gates 6
1.4. Counting With Wires 15
1.5. Math? 19

Part 2: Machinery **31**
2.1. Storing a Bit 31
2.2. Public Transport? 40
2.3. Organizing Data 46
2.4. The ALU 55
2.5. The Big Picture 58

Part 3: Execution **63**
3.1. Instructions Are Numbers 63
3.2. The Not-So-Clever Control Unit 68
3.3. Decode & Execute 72
3.4. Finally, The Fun Part 82
3.5. The End 85

Appendix: Quick Reference **87**

Further Reading **89**

Acknowledgments **90**

print at 100% scale (“Actual Size”) — cut along the black line — true book size 6 × 9 in

Part 1: Foundations

You can hear the phrase "computers think in 1s and 0s" a hundred times and still not understand how a computer actually works. By itself, this explains basically nothing. Sure, a wire can be high or low. Sure, a light can be on or off. But how does that become addition?

How does that become memory?

How does that become a program sitting in RAM, one instruction after another, telling a machine what to do?

Some resources stay extremely high-level, so you never really understand how a CPU actually works. The deeper resources are amazing, but they are long, dense, and intimidating. And frankly, for someone who doesn't want that level of detail, a lot of it can often feel like too much.

My goal is to help you understand what is going on under the hood, without exploding your brain or eating weeks of time.

We start with a simple circuit turning a light bulb on and off, then work our way through logic gates, memory, and the basic circuits underneath them.

The key point is that nothing here is smart in isolation. A CPU is not one hard idea. It is a very tall pile of simple ones.

print at 100% scale ("Actual Size") — cut along the black line — true book size 6 × 9 in

1.1. How to Read This

- If a diagram is hard to understand, click it and step through each frame one by one using the arrow keys, with the provided descriptions. This only works if you are reading on my website, bytecolony.computer.
- Don't try to "memorize" every layout. Focus on the mental models, concepts, and what part each piece has to play.
- If a section feels dense or hard to understand, follow the diagrams first and try to get a feel for what is happening.

1.2. Circuits & Switches

First, we need the basics of how electricity and circuits work. Here is a simple circuit:

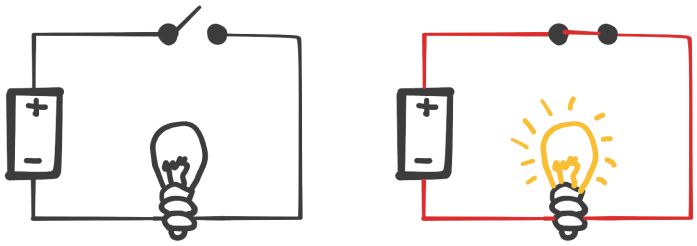


Diagram 1.2. A basic circuit with a battery, switch, and bulb.

The wire connected to the - of the battery is being driven low, and the wire connected to the + of the battery is being driven high. This difference in voltage causes current to flow through the filament of the light bulb, turning the light on. But current can only flow when the loop is complete, from + through the bulb to -.

If the loop is broken, nothing flows. A switch is simply a controlled break in the loop, allowing us to break and complete the loop whenever we want. And a light bulb is just a simple light bulb. It glows when current flows through the filament.

print at 100% scale ("Actual Size") — cut along the black line — true book size 6 × 9 in

path for that frame. I think it makes the current path easier to follow visually, and so the concept is just easier to understand, especially for beginners.

This is how the ground symbol looks:

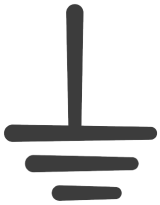
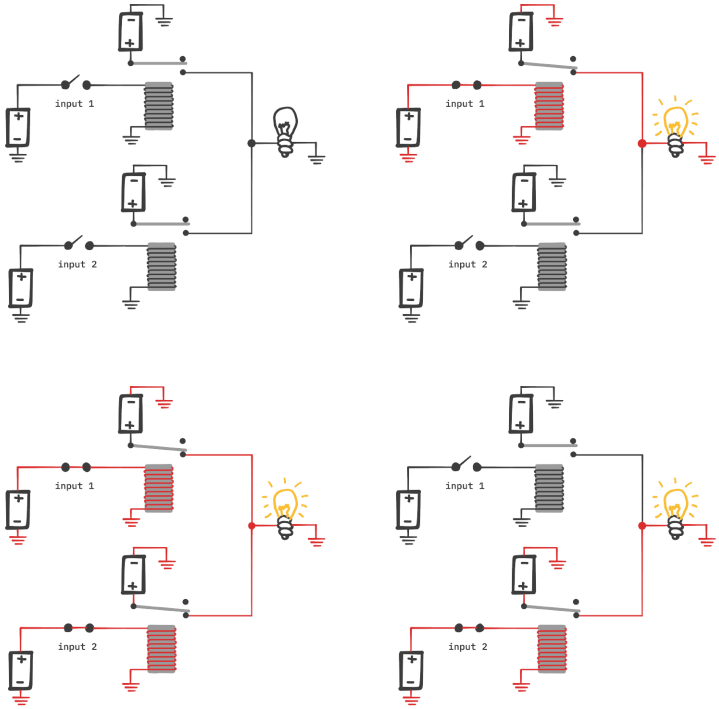


Diagram 1.7. The ground symbol.

Now here is the OR gate:



print at 100% scale (“Actual Size”) — cut along the black line — true book size 6 × 9 in

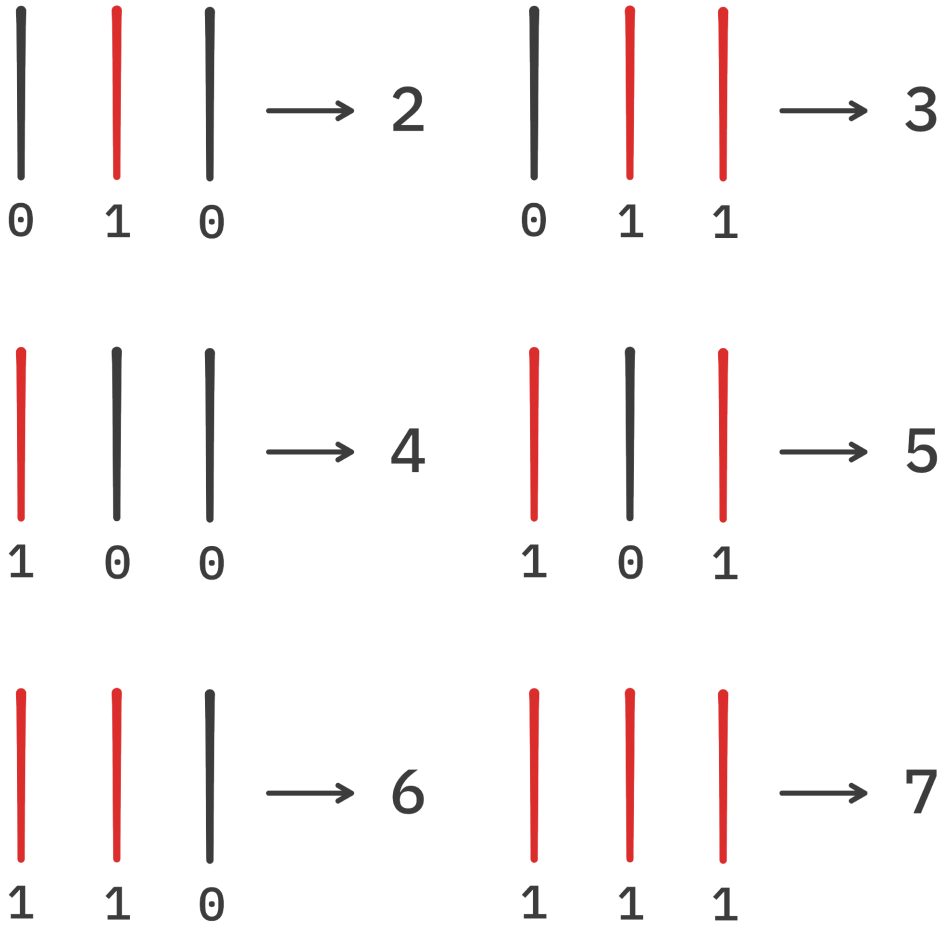


Diagram 1.17. States with 3 wires.

We can represent 8 numbers just like this. Keep going to 8 wires and there are `2^8`, or 256, possible patterns. A group of 8 bits is called a byte. So if we use those patterns to represent non-negative numbers, one byte can represent 0 through 255.

But why does `010` mean 2? Why does `101` mean 5? Is it just randomly assigned? Not exactly. To understand this, let's take a quick detour to decimal, a.k.a. base ten.

print at 100% scale ("Actual Size") — cut along the black line — true book size 6 × 9 in

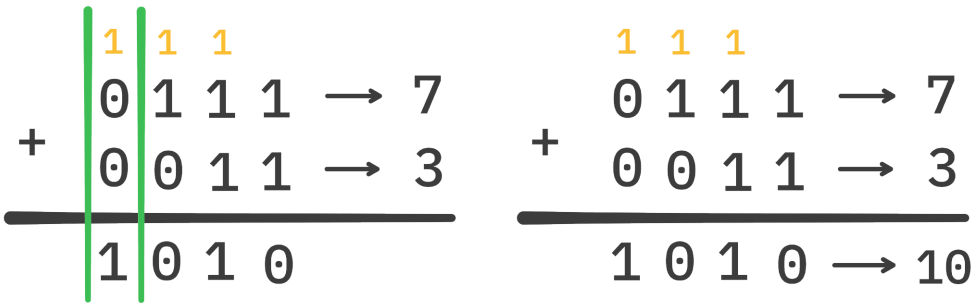


Diagram 1.23. Binary addition.

`1 + 1` gives `10`, which is binary for 2.

So the sum bit for that column is `0`, and the carry is `1`.

`1 + 1 + 1` gives `11`, which is binary for 3. So the sum bit is `1`, and the carry is `1`, and so on.

How do we build a circuit using logic gates that performs this standard addition algorithm?

Well, let's start with the rightmost column. If we think about it, all the possible states are:

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

So just `0 + 0`, `1 + 0`, `1 + 1`, or `0 + 1`. If we can make a tiny circuit that takes two inputs, and produces two outputs that match these combinations, we have added the first column.

This is called a half adder. A half adder adds two bits, but it does not handle a carry-in value. That is the job of a full adder.

Let's first build this half adder. Let's start by computing the sum, not the carry-out. This is what we want our circuit to do:

print at 100% scale ("Actual Size") — cut along the black line — true book size 6 × 9 in

When the enable wire turns on, `Data` gets stored in `Q`. Or in other words, when we turn the enable wire on, `Q` mirrors `D`. Then when we turn `E` off, `Q` stays stable with whatever `D` was last.

This type of latch is called a D latch, D meaning data. It can be made using the SR latch and a few extra logic gates.

It basically checks: if `D` is true and `E` is true, `SET` is true, and if `D` is false and `E` is true, `RESET` is true. That's it, so let's not worry about the exact implementation.

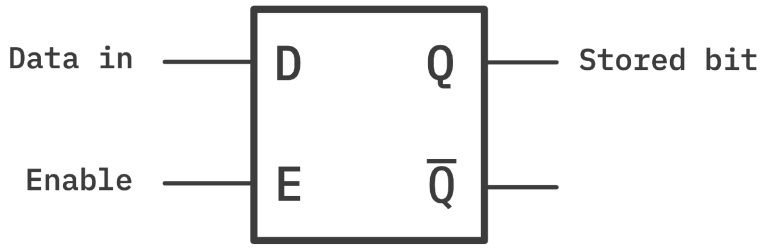


Diagram 2.2. D latch.

But we have a problem. Let's say we now try to use 8 of these D latches to hold the result from our adder, which would then feed back into the input for our accumulator (that counting circuit from last section, I'll call it an accumulator from now on). It still wouldn't work.

Here is the problem: say we have the enable wire hooked up to a button. When that button is pressed down, the enable wire is on, so `Q=D` for that time. But if `Q` feeds back into the adder, and the result of the adder `D` changes quickly enough, `Q` can change again, jumping unpredictably based on how long we hold that button for.

If we want the accumulator to work correctly, we need the enable wire to turn on for an instant and then turn back off. That is just hard to do.

print at 100% scale ("Actual Size") — cut along the black line — true book size 6 × 9 in

This system is known technically as `RAM`: Random Access Memory. It is called `RAM` because when the CPU wants to access a slot, it just knows the number and can access any slot at will. It is not like flipping through a book looking for the right page. It is more like grabbing a book from a bookshelf, where you already know exactly where the book sits.

Now let's think about exactly what we would want this `RAM` chip to do.

- `address`: the slot we wish to access
- `WRITE`: whether we want to write a value to this address
- `OUT`: whether we want to output the value onto the bus
- `data in`: the value we would like to write
- `data out`: the data output line

To be clear, `WRITE` and `OUT` are control signals, so just 1 input wire each.

For this demo `RAM`, `address` is only 4 input wires. `data in` and `data out` carry bytes and are both connected directly to the common bus. This only works if no other part is driving the bus when `OUT` is enabled.

So we are going to build a minuscule 16-byte `RAM`: 16 addresses, with each address storing one byte. This design can be scaled up easily. Our address will be 4 bits long, because `2^4` is 16.

We could do this as a tall stack of 16 registers, but a grid is nicer. So we will split the 4-bit address in half. The bottom two bits pick the row, and the top two bits pick the column:

```
top 2 bits    = column
bottom 2 bits = row
```

Two bits can choose 4 values, so this gives us a 4×4 grid of memory slots. That is 16 total bytes! Once the address selects a slot, two things can happen:

- If `WRITE` turns on, the selected slot stores `data in`.
- If `OUT` is on, the selected slot drives its stored byte onto `data out`.

Let's start with building a simple decoder. This decoder will take 2 bits of our address and, based on that number, turn on exactly one of its 4 outputs.

In the diagram, the top bit is the bigger bit (the 2's place) and the bottom bit is the smaller bit (the 1's place).

print at 100% scale ("Actual Size") — cut along the black line — true book size 6 × 9 in

E inputs, before we made it edge-triggered? While its enable was on, Q simply equaled D. No edge-triggering or anything like that. This new register is a D latch with the enable permanently on: it never really holds anything. Its output just follows its input.

But like our newer registers, its output goes through tri-state buffers with an OUT control signal, so we still decide when it outputs. So in total: a data input that is always being stored, an OUT control signal, and an output.

Also, you'll see me feeding two buses into a single logic gate. "How does that work?", you might think. Well, there are really just eight gates, one per bit. One gate takes A0 and B0, the next takes A1 and B1, and so on. Eight output wires, which is just another bus.

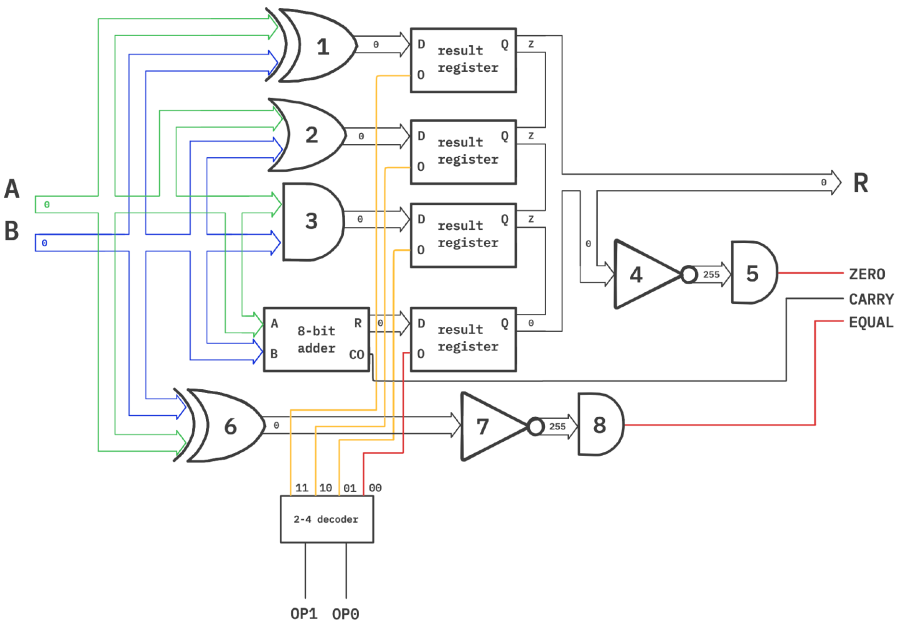


Diagram 2.27. ALU internals.

At its crux, the ALU works by routing A and B into all 4 operations at once, in this case XOR, OR, AND, and ADD. We store each of the results in a corresponding result register.

print at 100% scale ("Actual Size") — cut along the black line — true book size 6 × 9 in

#	Binary	Mnemonic	Bytes	Action
0	`0000`	`NOP`	1	do nothing
1	`0001`	`LOAD A, n`	2	next byte -> A
2	`0010`	`LOAD A, [addr]`	2	RAM slot -> A
3	`0011`	`STORE A, [addr]`	2	A -> RAM slot
4	`0100`	`MOV B, A`	1	copy A into B
5	`0101`	`MOV A, B`	1	copy B into A
6	`0110`	`JMP addr`	2	addr -> PC
7	`0111`	`JE addr`	2	if `EQUAL` flag: addr -> PC
8	`1000`	`ADD`	1	A = A + B
9	`1001`	`AND`	1	A = A AND B
10	`1010`	`OR`	1	A = A OR B
11	`1011`	`XOR`	1	A = A XOR B
12	`1100`	`JC addr`	2	if `CARRY` flag: addr -> PC
13	`1101`	`JZ addr`	2	if `ZERO` flag: addr -> PC
14	`1110`	`HALT`	1	stop the clock

Table 3.1. The full instruction set.

The number for each instruction is called its opcode.

Let's write a tiny program. We store `00000001` in address 0 of `RAM`. Then store `00000101` in address 1 of `RAM`. Then store `00000011` in address 2, then `11111111` in address 3, and finally `00001110` in address 4. Our `RAM` looks like this:

```
0: 00000001
1: 00000101
2: 00000011
3: 11111111
4: 00001110
```

If `PC` starts at 0, which it does, then what do you think this program would do?

The first byte is `00000001`, which is 1, and the table says 1 is `LOAD A, n`: load the next byte into register `A`. The next byte is `00000101`, binary for 5, so those two bytes make up the instruction:

```
LOAD A, 5
```


print at 100% scale ("Actual Size") — cut along the black line — true book size 6 × 9 in

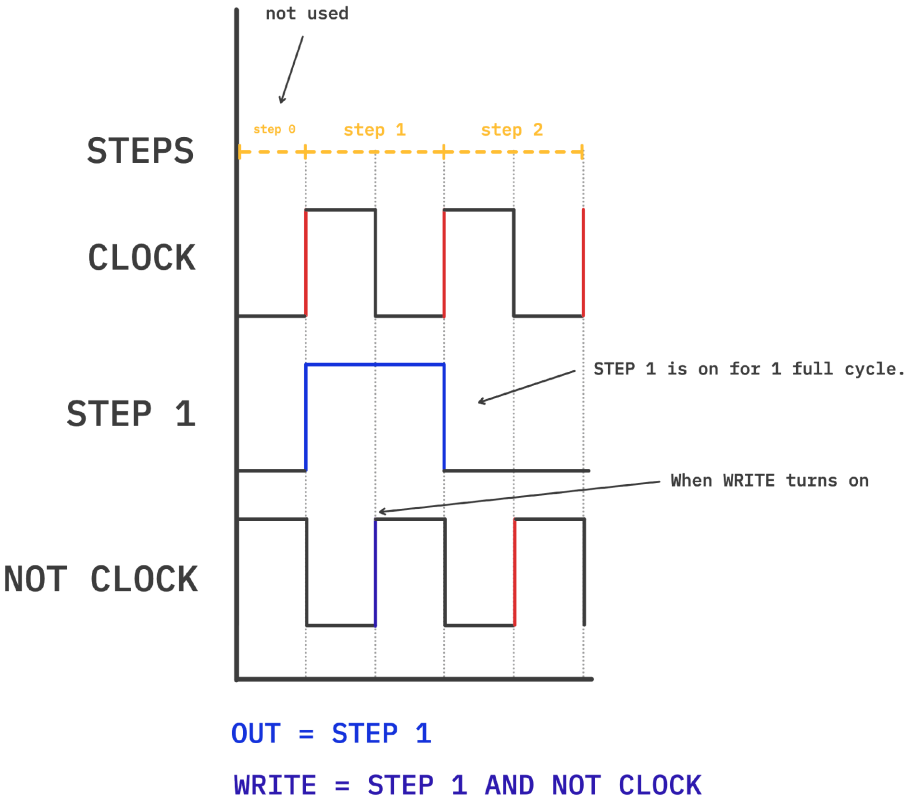


Diagram 3.2. Micro-step timing diagram.

Purple marks when `W` turns on, and blue is when `O` is on.

Each step changes on a clock edge, and each step is a signal, 1 full clock cycle long, until the next rising edge.

`O` is directly connected to that step line. If we enable `O` on step 1, the bus is driven for that full cycle.

`W` is ANDed with `NOT CLOCK`, like `step 1 AND NOT CLOCK`. The thing is, step and `NOT CLOCK` don't come on at the same time. The step line is already on, and then halfway through the cycle `NOT CLOCK` goes up, and that is when the AND gate's output goes up. That is the edge `W` needs, right in the middle, after `O` has had some time to settle.

print at 100% scale ("Actual Size") — cut along the black line — true book size 6 × 9 in

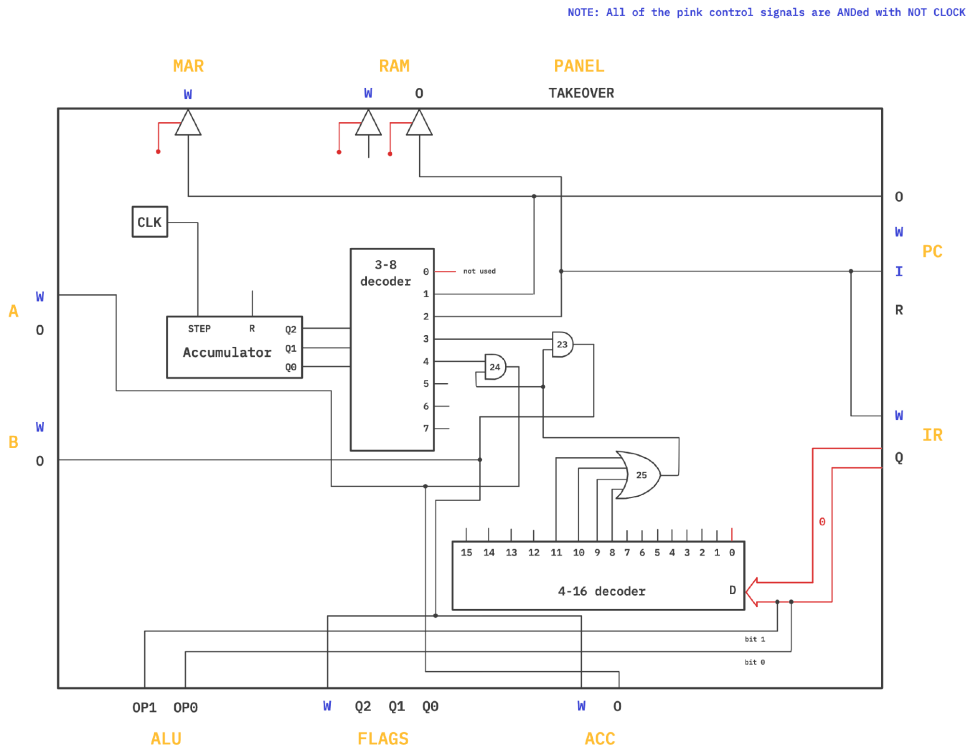


Diagram 3.7. The ALU instructions.

Again I am leaving out all the instructions apart from these 4, and again this wiring is a mess.

All you need to know is that the bottom 2 bits of the opcode are tapped out and connected directly to the ALU's OP bits. And the rest of the logic just checks if it is an ALU instruction and if so, on the appropriate steps, it enables the correct control wires according to the table above.

One last thing: HALT. This instruction just stops the clock. The way we do this is by using an SR latch, whose Q is inverted and then ANDed with the clock signal itself. So, on step 3 of instruction number 14, we just set that latch, and boom! We froze the computer.

print at 100% scale ("Actual Size") — cut along the black line — true book size 6 × 9 in

Appendix: Quick Reference

The instruction set

#	Binary	Mnemonic	Bytes	Action
0	`0000`	`NOP`	1	do nothing
1	`0001`	`LOAD A, n`	2	next byte → A
2	`0010`	`LOAD A, [addr]`	2	RAM slot → A
3	`0011`	`STORE A, [addr]`	2	A → RAM slot
4	`0100`	`MOV B, A`	1	copy A into B
5	`0101`	`MOV A, B`	1	copy B into A
6	`0110`	`JMP addr`	2	addr → PC
7	`0111`	`JE addr`	2	if `EQUAL` flag: addr → PC
8	`1000`	`ADD`	1	A = A + B
9	`1001`	`AND`	1	A = A AND B
10	`1010`	`OR`	1	A = A OR B
11	`1011`	`XOR`	1	A = A XOR B
12	`1100`	`JC addr`	2	if `CARRY` flag: addr → PC
13	`1101`	`JZ addr`	2	if `ZERO` flag: addr → PC
14	`1110`	`HALT`	1	stop the clock

print at 100% scale (“Actual Size”) — cut along the black line — true book size 6 × 9 in

About the Author



Ekansh Goenka (b. 2013) wrote this book in 2026, over about six months. He drew all 71 diagrams by hand and designed its 8-bit CPU from scratch.

He wrote it because once he understood how a CPU works in detail, it turned out to be simpler than most people think. The resources he found either skipped the juicy details, so you never really got it, or were so long and intimidating that most people never started.

The web version is free at bytecolony.computer, with the animated diagrams.